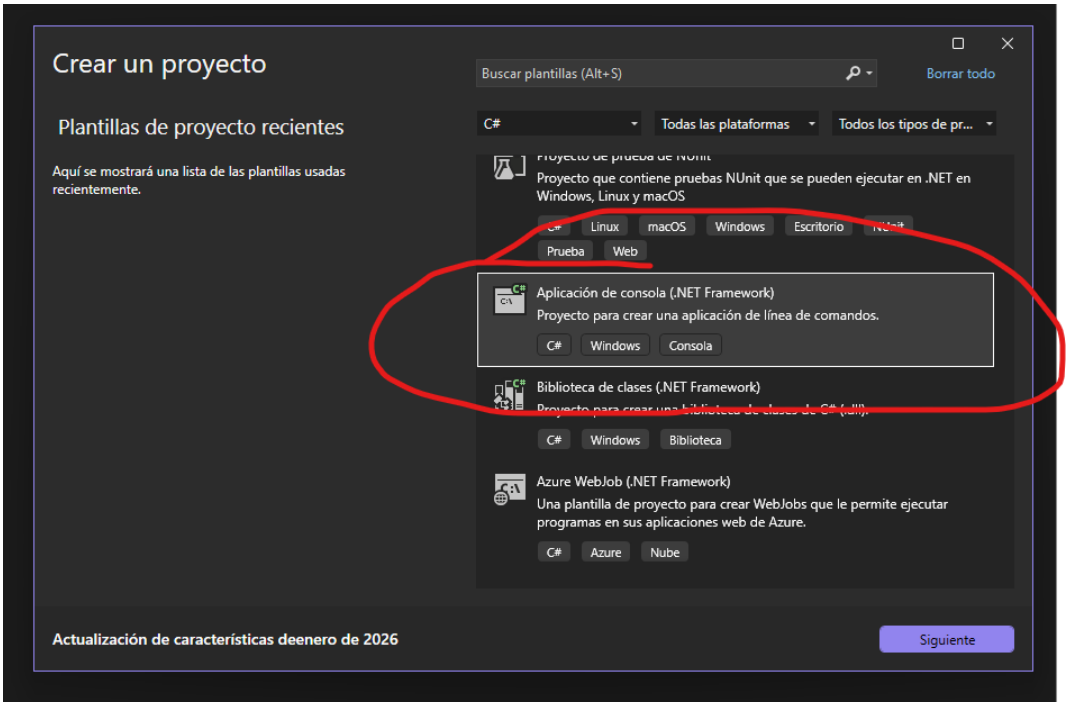
EJERCICIO. LAS FUNCIONES EN C#

El objetivo de este ejercicio es seguir programando con .NET y C# y aprender LAS FUNCIONES

Creamos un proyecto nuevo



Nombre del proyecto LAS FUNCIONES

1) Sumar dos números

Crea un método Sumar que reciba dos int y devuelva su suma.

Solución propuesta

```
static int Sumar(int a, int b) => a + b;
```

2) Área de un círculo

Crea un método AreaCirculo que reciba el radio (double) y devuelva el área. Usa Math.PI.

Solución propuesta

```
static double AreaCirculo(double radio)
```

```
{
    if (radio < 0) throw new ArgumentException("El radio debe ser >= 0");
    return Math.PI * radio * radio;
}
```

3) ¿Es par?

Crea un método EsPar que reciba un int y devuelva true si es par.

Solución propuesta

```
static bool EsPar(int n) => n % 2 == 0;
```

4) Máximo de tres números (sobrecarga opcional)

Crea un método MaxDeTres que reciba tres int y devuelva el mayor.

Solución propuesta

```
static int MaxDeTres(int a, int b, int c)
{
    int maxAB = a > b ? a : b;
    return maxAB > c ? maxAB : c;
}
```

5) Contar vocales

Crea un método ContarVocales que reciba un string y devuelva cuántas vocales tiene (a,e,i,o,u).

Solución propuesta

```
static int ContarVocales(string texto)
{
    if (texto is null) throw new ArgumentNullException(nameof(texto));
    int contador = 0;
    foreach (char ch in texto.ToLower())
    {
        if ("aeiou".Contains(ch)) contador++;
    }
    return contador;
}
```

6) TryParse personalizado (out)

Crea un método TryLeerEdad que reciba un string y devuelva bool. Si puede, devuelve la edad por out.

Solución propuesta

```
static bool TryLeerEdad(string texto, out int edad)
{
    if (int.TryParse(texto, out edad) && edad >= 0)
        return true;

    edad = 0;
    return false;
}
```

7) Intercambiar valores (ref)

Crea un método Swap que reciba dos int por ref y los intercambie.

Solución propuesta

```
static void Swap(ref int a, ref int b)
{
    int temp = a;
    a = b;
    b = temp;
}
```

8) Sumar muchos números (params)

Crea un método SumarMuchos con params int[] y devuelve la suma total.

Solución propuesta

```
static int SumarMuchos(params int[] numeros)
{
    int total = 0;
    foreach (var n in numeros) total += n;
    return total;
}
```

9) Factorial (recursivo)

Crea un método Factorial que reciba int n (>=0) y devuelva n!.

Solución propuesta

```
static long Factorial(int n)
{
    if (n < 0) throw new ArgumentException("n debe ser >= 0");
    if (n == 0) return 1;
```

```
        return n * Factorial(n - 1);
    }
```

10) Mini reto: limpiar y formatear un nombre (métodos encadenados)

Crea un método FormatearNombre que reciba nombre y apellidos, quite espacios de sobra y devuelva "Apellidos, Nombre".

Solución propuesta

```
    static string FormatearNombre(string nombre, string apellidos)
    {
        if (nombre is null) throw new ArgumentNullException(nameof(nombre));
        if (apellidos is null) throw new ArgumentNullException(nameof(apellidos));

        string n = nombre.Trim();
        string a = apellidos.Trim();

        if (n.Length == 0 || a.Length == 0)
            throw new ArgumentException("Nombre y apellidos no pueden estar vacíos");

        return $"{a}, {n}";
    }
```

Apéndice: plantilla de pruebas rápidas (Main)

```
    using System;

    class Program
    {
        static void Main()
        {
            Console.WriteLine(Sumar(2, 3));
            Console.WriteLine(AreaCirculo(2));
            Console.WriteLine(EsPar(10));

            int x = 5, y = 9;
            Swap(ref x, ref y);
            Console.WriteLine($"x={x}, y={y}");
```

```
        Console.WriteLine(SumarMuchos(1, 2, 3, 4));
        Console.WriteLine(Factorial(5));

        Console.WriteLine(FormatearNombre("  Ana  ", "  García López  "));
    }

    // Pega aquí tus métodos...
}
```